RESEARCH REPORT

NOVA AI CHATBOT

A WEB-BASED AI ASSISTANT USING PYTHON AND FLASK



Submitted By:

VF MOHAMMED FAYAS

24BCA1354

ARTIFICAL INTELLIGENCE DEVOPS AND FULL STACK DEVELOPMENT

5TH SEMESTER

Submitted To:

MS. DIVYA

BACHELOR OF COMPUTER APPLICATION

YENEPOYA UNIVERSITY

2026

ABSTRACT

Artificial Intelligence (AI) is increasingly used to help people find information, learn concepts, generate ideas, and complete everyday tasks. A conversational AI chatbot provides a natural-language interface through which users can ask questions and receive responses. This report presents Nova AI Chatbot, a web-based assistant developed with HTML, CSS, JavaScript, Python, and Flask. The user interface provides a chat workspace, a message composer, starter prompts, and controls for beginning or clearing a conversation.

The backend receives user messages through a Flask route and forwards the prompt to a language-model service through an API client. Configuration values, including the API key and model settings, are stored in environment variables rather than being placed directly in the source code. The response is returned to the browser and displayed in the conversation area. The report describes the problem statement, objectives, related work, system architecture, methodology, implementation, testing considerations, applications, limitations, and future enhancements.

The current project is an API-integrated chatbot application; it does not claim to train a large language model from scratch. Response quality, availability, and cost depend partly on the configured model provider and its service limits.

Keywords: Artificial Intelligence, Chatbot, Natural Language Processing, Python, Flask, JavaScript, Large Language Model, Web Application.

TABLE OF CONTENTS

Section	Title	Page No.
Chapter 1	Introduction	
1.1	Background of the Topic	
1.2	Problem Statement	
1.3	Objectives	
1.4	Scope of the Study	
1.5	Importance of the Project	
Chapter 2	Literature Review	
2.1	Conversational Systems	
2.2	Language Models and Transformers	
2.3	Instruction Following and Limitations	
2.4	Research Gap	
Chapter 3	Proposed System and Methodology	
3.1	System Overview	
3.2	Technology Stack	
3.3	System Architecture	
3.4	Working Principle	
3.5	User Interface Design	
3.6	Backend Processing	
3.7	Security and Configuration	
3.8	System Flow	
Chapter 4	Implementation and Applications	
4.1	Frontend Implementation	
4.2	Backend Implementation	
4.3	Screenshot of the Project	
4.4	Applications	
Chapter 5	Advantages, Limitations and Testing	
5.1	Advantages	
5.2	Limitations	
5.3	Testing Plan	
Chapter 6	Future Scope	
Chapter 7	Conclusion	
References	References	
Appendix	Setup and Run Instructions	

CHAPTER 1: INTRODUCTION

1.1 Background of the Topic

A chatbot is a software application designed to communicate with users through text or other conversational interfaces. Earlier chatbots often depended on fixed rules and scripted responses. Modern AI chatbots can use language models to interpret a prompt and generate a response in natural language. When integrated into a web application, a chatbot can be accessed through a familiar browser interface without requiring users to install a separate desktop program.

Nova AI is designed as a personal AI assistant interface. The page uses a dark visual theme, a left navigation area, a central welcome message, suggestion cards, and a message composer. The application combines standard frontend technologies with a Python backend so that browser interactions can communicate with an external language-model API.

1.2 Problem Statement

Users may need a simple, accessible interface to ask questions, explore ideas, or learn technical concepts. Building a conversational interface also involves coordinating frontend events, server-side request handling, API configuration, and response rendering. The project addresses the problem of creating a clean web interface that sends a user's message to an AI service and presents the returned answer in a readable chat format.

1.3 Objectives

- To develop a responsive chat interface using HTML, CSS, and JavaScript.
- To connect the browser interface to a Python Flask backend.
- To send user prompts to a configured language-model API and display responses.
- To provide clear interaction controls such as new chat and clear chat.
- To keep API configuration in environment variables rather than hard-coding secrets.
- To document the design, workflow, testing plan, limitations, and future improvements.

1.4 Scope of the Study

The scope covers the web interface, message submission, server-side API integration, response display, basic error handling, and local development setup. It also discusses safe configuration practices and potential extensions. The exact capabilities of the assistant depend on the selected

model and provider. Features such as accounts, saved conversation history, file uploads, voice input, and model training are not assumed to be implemented unless added separately.

1.5 Importance of the Project

The project provides practical experience in connecting a frontend to a backend service. It demonstrates event handling in JavaScript, HTTP requests, Flask routing, environment configuration, and integration with an AI API. These skills are useful for developing many real-world applications, including support assistants, learning tools, internal knowledge interfaces, and productivity applications.

CHAPTER 2: LITERATURE REVIEW

2.1 Conversational Systems

Conversational systems have evolved from rule-based dialogue programs to statistical and neural approaches. Rule-based systems are predictable within their predefined flows but can struggle with unexpected user requests. Neural systems learn patterns from data and can respond more flexibly, although they may produce incorrect or unsupported information.

2.2 Language Models and Transformers

The Transformer architecture introduced attention mechanisms that support efficient sequence modelling and became foundational to many modern language models. Large language models are trained on extensive text data to predict and generate language. Depending on their training and configuration, they can support summarization, question answering, explanation, coding assistance, and other language tasks.

2.3 Instruction Following and Limitations

Instruction-tuning and feedback-based methods have been used to make language models more helpful and aligned with user instructions. Nevertheless, generated responses can contain hallucinations, outdated information, bias, or errors. A practical chatbot should communicate uncertainty where appropriate and encourage verification for important decisions.

2.4 Research Gap

Research often focuses on model architecture, training, and benchmark performance. A student web project also needs to address the engineering work required to make a model accessible: a usable interface, API communication, error states, secret management, and understandable

output. Nova AI focuses on this integration layer rather than claiming to introduce a new language model.

Selected Research Summary

Vaswani et al. (2017), Attention Is All You Need. Introduced the Transformer architecture using attention mechanisms; foundational to many later language models.

Brown et al. (2020), Language Models are Few-Shot Learners. Studied large-scale language models and their ability to perform tasks from examples in prompts.

Ouyang et al. (2022), Training Language Models to Follow Instructions with Human Feedback. Explored instruction following using supervised demonstrations and human feedback.

Ji et al. (2023), Survey of Hallucination in Natural Language Generation. Reviewed hallucination challenges and approaches to evaluating generated language.

Zhao et al. (2023), A Survey of Large Language Models. Surveyed model development, capabilities, evaluation, and open challenges.

CHAPTER 3: PROPOSED SYSTEM AND METHODOLOGY

3.1 System Overview

Nova AI follows a client-server architecture. The browser renders the interface and handles user interactions. JavaScript collects the prompt and sends a request to a Flask endpoint. The Flask backend validates the request, reads configuration from environment variables, calls the configured language-model service, and returns a response to the browser.

3.2 Technology Stack

- HTML: page structure and accessible interface elements.
- CSS: layout, dark theme, spacing, borders, responsive styling, and visual feedback.
- JavaScript: message submission, chat updates, loading states, and client-server requests.
- Python: backend application logic.
- Flask: HTTP routes and serving the web application.
- OpenAI-compatible SDK/API client: communication with the configured language-model service.

- python-dotenv: loading local environment variables during development.
- Environment file (.env): stores API key and model configuration; must not be committed to public version control.

3.3 System Architecture

- User enters a message in the browser.
- JavaScript validates and submits the message to the Flask backend.
- Flask reads the model/provider configuration and forwards the prompt to the API.
- The model provider generates a response and returns it to Flask.
- Flask sends the reply to the browser as a structured response.
- JavaScript displays the reply in the conversation interface.

3.4 Working Principle

The application starts when the Flask development server is launched. The browser loads the HTML template and static CSS/JavaScript files. When a user sends a message, the frontend adds the user's text to the conversation and can show a loading indicator. The request is processed on the server, which contacts the configured API. When a response arrives, the loading state is replaced with the assistant's answer. If the API request fails, the application should show a clear error rather than silently appearing to hang.

3.5 User Interface Design

The interface shown in the project screenshot uses a dark navy background with violet accents. A sidebar contains the Nova AI identity, a New Chat control, and a short assistant description. The main workspace includes a top bar, a welcome heading, four suggested prompt cards, a message input area, a send control, a clear-chat control, and a footer indicating the technologies used. The design prioritizes visual hierarchy and a focused conversation area.

3.6 Backend Processing

The Flask application receives the frontend request at a defined route, extracts the user message, and passes it to the API client. The backend should check for missing or empty prompts, handle provider errors, and return JSON with a consistent response field. The frontend then updates the chat. The exact endpoint and response property should match the implementation in app.py and script.js.

3.7 Security and Configuration

API keys are credentials and should remain on the server. They should be read from environment variables, excluded from public repositories, and rotated if accidentally exposed. Client-side JavaScript should not contain the provider secret. If Markdown responses are rendered as HTML, the rendered output should be sanitized before insertion into the page. Production deployments should also consider rate limiting, input size limits, logging without sensitive content, and HTTPS.

3.8 System Flow

START

↓

Open Nova AI in browser

↓

Enter a message

↓

JavaScript sends request to Flask

↓

Validate request and call language-model API

↓

Receive response or error

↓

Display assistant response / error

↓

Continue chatting or clear chat

↓

END

CHAPTER 4: IMPLEMENTATION AND APPLICATIONS

4.1 Frontend Implementation

The HTML template defines the main page structure and controls. CSS provides the dark theme, sidebar and main-panel layout, suggestion cards, rounded message composer, and responsive presentation. JavaScript manages events such as sending a message, displaying user and assistant messages, clearing the conversation, and showing loading feedback.

4.2 Backend Implementation

The Python Flask application serves the web page and exposes the message-processing endpoint. It uses environment configuration to determine the API key and model/provider settings. The external model service performs response generation; Flask acts as the bridge between the browser and that service.

4.3 Screenshot of the Project

Figure 4.1 shows the current Nova AI Chatbot home screen provided for this report. It illustrates the navigation sidebar, welcome state, suggested prompt cards, message composer, and visual styling.

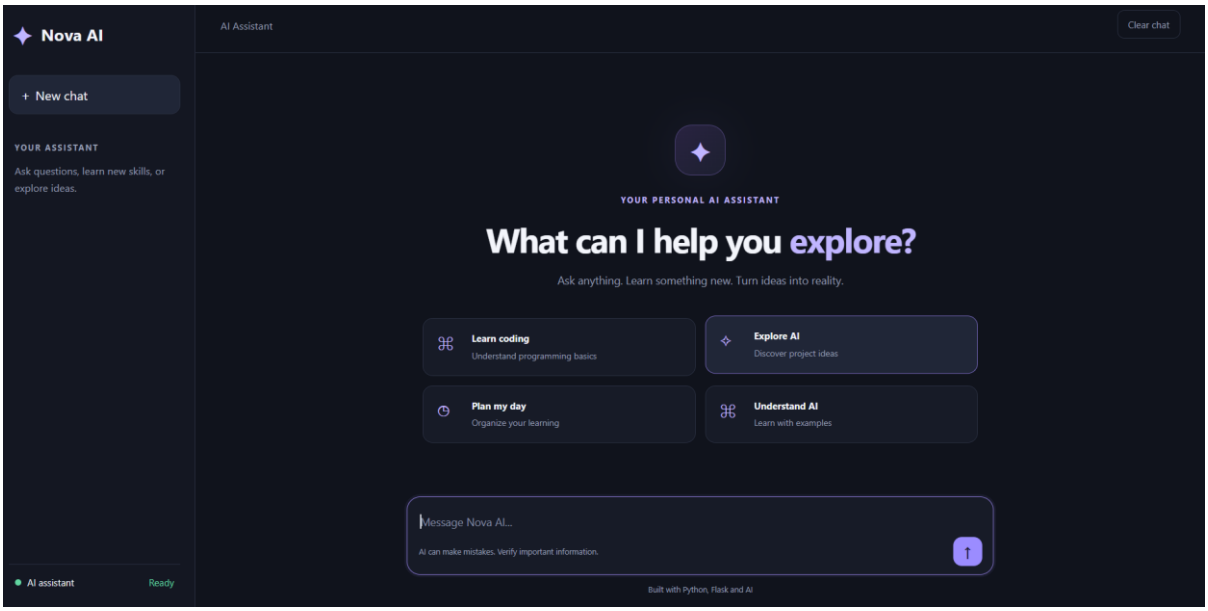


Figure 4.1: Nova AI Chatbot user interface (project screenshot supplied by the developer)

Note: This is the screenshot supplied for documentation. Additional screenshots of a successful response, error handling, and mobile layout can be inserted after capturing those states from the running application. The report does not fabricate these unprovided screenshots.

4.4 Applications

- Learning support: asking for explanations, examples, and study assistance.
- Programming support: discussing code, debugging ideas, and technical concepts.
- Idea exploration: brainstorming project concepts and organizing information.
- Productivity: drafting outlines, plans, and summaries.
- Customer support prototypes: serving as a starting point for a domain-specific assistant.

CHAPTER 5: ADVANTAGES, LIMITATIONS AND TESTING

5.1 Advantages

- Accessible through a standard web browser.
- Clear and modern chat-oriented interface.
- Separation between frontend and backend responsibilities.
- Can use a hosted model without training a model from scratch.
- Configuration can be changed through environment variables.
- Provides practical experience with web development and API integration.

5.2 Limitations

- The application depends on the availability and limits of the configured API provider.
- API usage may incur costs or have rate limits depending on provider and plan.
- AI responses may be inaccurate, incomplete, or misleading.
- The application requires an internet connection for a hosted model API.
- Conversation persistence, authentication, and advanced privacy controls are not assumed to exist.
- The current screenshot documents the home screen only; other states need separate verification.

5.3 Testing Plan

Test	Procedure	Expected Result
Page load	Open the local application URL	Home screen and styling load correctly
Empty message	Send an empty prompt	Input is rejected or no empty request is sent
Valid prompt	Submit a normal question	User message and model reply are displayed
Loading state	Submit a prompt and observe while waiting	Loading feedback appears and is removed after completion
Provider/API failure	Use an invalid key or simulate provider failure in a safe test environment	A readable error is shown without exposing secrets

Clear chat	Click Clear chat	Visible conversation is cleared as intended
Responsive layout	Resize browser or use device emulation	Controls remain usable at narrower widths
Secret handling	Inspect frontend source and repository configuration	API key is not exposed in browser code or committed files

CHAPTER 6: FUTURE SCOPE

The project can be improved through a number of extensions. Conversation history could be stored in a database with clear user controls and appropriate privacy safeguards. User authentication could support individual workspaces. A model selector and configurable system instructions could make the application more flexible. Streaming responses could reduce perceived waiting time. File upload, speech input, and text-to-speech could broaden accessibility. A production deployment should include HTTPS, rate limiting, robust input validation, monitoring, and secure secret management. Automated tests and a deployment pipeline would also improve reliability.

CHAPTER 7: CONCLUSION

Nova AI Chatbot demonstrates how a browser-based interface can be connected to a hosted language model through a Python Flask backend. The frontend provides a chat workspace with navigation, prompt suggestions, and a message composer. The backend mediates requests to the model provider and returns generated responses to the browser. This structure gives the project a practical foundation for exploring conversational AI and full-stack development.

The project also highlights important engineering responsibilities: keeping API keys private, handling errors, sanitizing generated content when necessary, testing different user flows, and explaining that AI responses may be wrong. Nova AI is an API-integrated application rather than a model trained from scratch. With further work on testing, persistence, accessibility, security, and deployment, it can be developed into a more complete assistant application.

REFERENCES

1. Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems.

2. Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems.

3. Ouyang, L., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. Advances in Neural Information Processing Systems.

4. Ji, Z., et al. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys.

5. Zhao, W. X., et al. (2023). A Survey of Large Language Models. arXiv preprint.

6. Jurafsky, D., & Martin, J. H. Speech and Language Processing. Online textbook draft.

7. Flask Documentation. https://flask.palletsprojects.com/

8. OpenAI API Documentation. https://platform.openai.com/docs/

9. Python Documentation. https://docs.python.org/3/

10. MDN Web Docs: HTML, CSS, and JavaScript. https://developer.mozilla.org/

11. python-dotenv documentation. https://saurabh-kumar.com/python-dotenv/

12. OWASP Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/

APPENDIX A: SETUP AND RUN INSTRUCTIONS

The following steps describe a typical local development workflow. Adjust filenames and environment variable names to match the actual project files.

- Install a supported Python version.
- Open a terminal in the project folder.
- Create and activate a virtual environment if required.
- Install dependencies using: pip install -r requirements.txt
- Create a local .env file and add the API key and model/provider configuration expected by app.py.
- Ensure .env is excluded from version control and never share the API key in screenshots or reports.
- Start the application using the command configured for the project, commonly: python app.py
- Open http://127.0.0.1:5000 in a browser.
- Send a test prompt and confirm that a response appears.

- Stop the development server with Ctrl+C when finished.

APPENDIX B: PROJECT FILE STRUCTURE

```
Nova-AI/
├── app.py                Flask backend
├── .env                  Local API configuration (keep private)
├── requirements.txt      Python dependencies
├── templates/
│   └── index.html        Chat interface
└── static/
    ├── style.css         Interface styling
    └── script.js         Chat interactions
```

APPENDIX C: DOCUMENTATION NOTE

Before submission, replace all cover-page placeholders with the correct student, registration, class, faculty, department, and institution details. Update the table of contents and page numbers after final editing. Capture and add genuine screenshots of successful conversation output and any other implemented features. Verify every reference and ensure that the final report matches the actual source code and college requirements.